

Milestone 1 — Acceptance Evidence

A living record of the proof behind each acceptance criterion. Everything here is reproducible from the repo; CI re-proves it on every push.

Automated CI (Linux + PostgreSQL)

- **CI workflow:** [.github/workflows/ci.yml](https://github.com/KM99999/my-client/actions/workflows/ci.yml) — install → prisma generate → **migrate deploy** → typecheck → unit tests → **integration suite (reserve-race, expiry, webhook)** → concurrency/expiry/webhook proofs.
 - green run: <https://github.com/KM99999/my-client/actions/runs/28630825510>
- **Load workflow:** [.github/workflows/load.yml](https://github.com/KM99999/my-client/actions/workflows/load.yml) — boots the real Express + Prisma server on Linux, runs k6 (50 VUs) at one contended slot, then asserts **exactly one reservation won**.
 - green run: <https://github.com/KM99999/my-client/actions/runs/28630825547>

The integration suite runs the guarantees through the **real application code** (Prisma `reserveSlot` , `releaseReservation` , the webhook handler) on Linux — not just the standalone scripts.

Criterion → evidence

#	Client criterion	Evidence
1	No double booking under load	<code>verify:locking</code> — 50 concurrent × 20 rounds, exactly one winner each round; integration <code>reserve-race</code> ; k6 HTTP load test (load.yml) asserting exactly one reservation.
2	Duplicate/retried webhooks confirm once, charge once	<code>verify:webhook</code> — same event ×5 (sequential + concurrent) → one confirmation, one charge; out-of-order convergence; integration <code>webhook</code> .
3	Expired reservations auto-release	<code>verify:expiry</code> + integration <code>expiry</code> — expired→freed, not-due→held, paid→never freed; precise job and sweep paths.
4	Payment-at-expiry always consistent	<code>verify:webhook</code> race — 50 rounds always resolve to Paid+Approved or freed+Refunded, never paid-but-unbooked.
—	Reproducible setup	<code>prisma migrate deploy</code> from a clean DB (CI does this every run); docs/RUN_ONLINE.md + docs/README.md .

Reproduce locally

```
npm test                # unit
npm run test:integration # Prisma integration (needs a DB)
npm run verify:locking  # concurrency proof
npm run verify:expiry   # expiry proof
npm run verify:webhook   # idempotency + payment-at-expiry proof
```

Pending (external dependencies)

- Live Pix + card sandbox runs (needs AbacatePay keys) — recordings to be added here.
- Staging deployment URL (needs Railway access) — to be added here.